Assignment 1

Key Information

Academic Integrity

Your code will be checked against all other students with a similarity checker.

Anything you are able to google can be easily found by the teaching team and compared against your work.

In this assessment, you must not use generative artificial intelligence (genAI) to generate any materials or content in relation to the assessment task. This includes ChatGPT and GitHub Copilot.

You are responsible for all code submitted as a part of your assignment.

How can I avoid academic integrity issues?

- Never copy code from anywhere. If you learn something useful online, rewrite it from scratch. It's also the best way to make sure you have understood it. This includes not submitting code you have submitted in a prior assessment in this or another unit. Self-plagiarism is also a breach of academic integrity.
- If a fellow student asks you for a solution to a question, you can try to help them build their solution, but **do not give them yours**. Giving your solution is just as much of an academic Integrity breach as receiving it.
- Don't leave your devices unlocked and unattended. Properly securing your work is your responsibility.

Assignment design

There are 3 independent parts to this assignment.

Each part is designed to cover 1 week of content and take one week of personal work to complete.

At the end of every week but the last, there is a "checkpoint", at which point we expect you to have completed the tasks of the week.

Each part contains multiple tasks of increasing difficulty.

The tasks of a given week are built on top of each other, therefore you need to attempt them in the order we provide them. Even if you can write a program to solve the last task directly, you still need to write programs for the other tasks in order to pass the tests and be eligible to receive up to full marks for program quality.

The difficult tasks are designed to provide a challenge to most students, therefore it would be perfectly normal and acceptable if you did not complete every single task.

Tips on working with automated tests

Handling inputs

You can assume that the inputs provided by the tests conform to the specifications in the brief. This means that you won't need to check that the inputs are correct, whether they are provided by the user or by the automated tests. If you really think there is an error in the tests, then please report it on ed.

The automated tests use complex inputs that may make it hard to debug. We recommend you run your programs with simple inputs that you can manually check.

Make sure that the program you are writing actually expects input or not. If you use the `input()` function when unexpected, you will receive a cryptic `EOF error` from the automarker. If you see this, double check all of your inputs are correct.

Formatting your outputs

In your program, ensure that the format of the output is the same as the format of the examples provided. Every character counts, including spaces and new lines.

Printing new lines

The last character is the newline character `'\n'` in every output file in the automated tests.

```
print("This is on line 1...\nThis is on line two!")
```

The code above produces the output below.

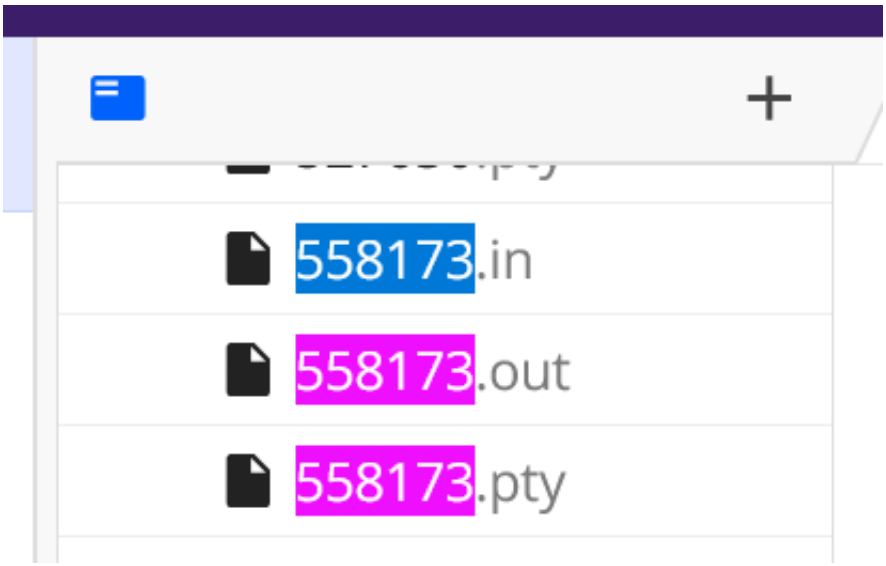
```
This is on line 1...
This is on line two!

```

Notice the that last line is empty. This is because the penultimate line is a `print`, which by default ends with a newline character `'\n'`.

Debugging failed tests

If your program does not pass all the tests, look up the ID of a test that fails:

Then you can look for the test in your scaffold (including with ctrl+F):



There are three files for each test under the `public` directory of your scaffold:

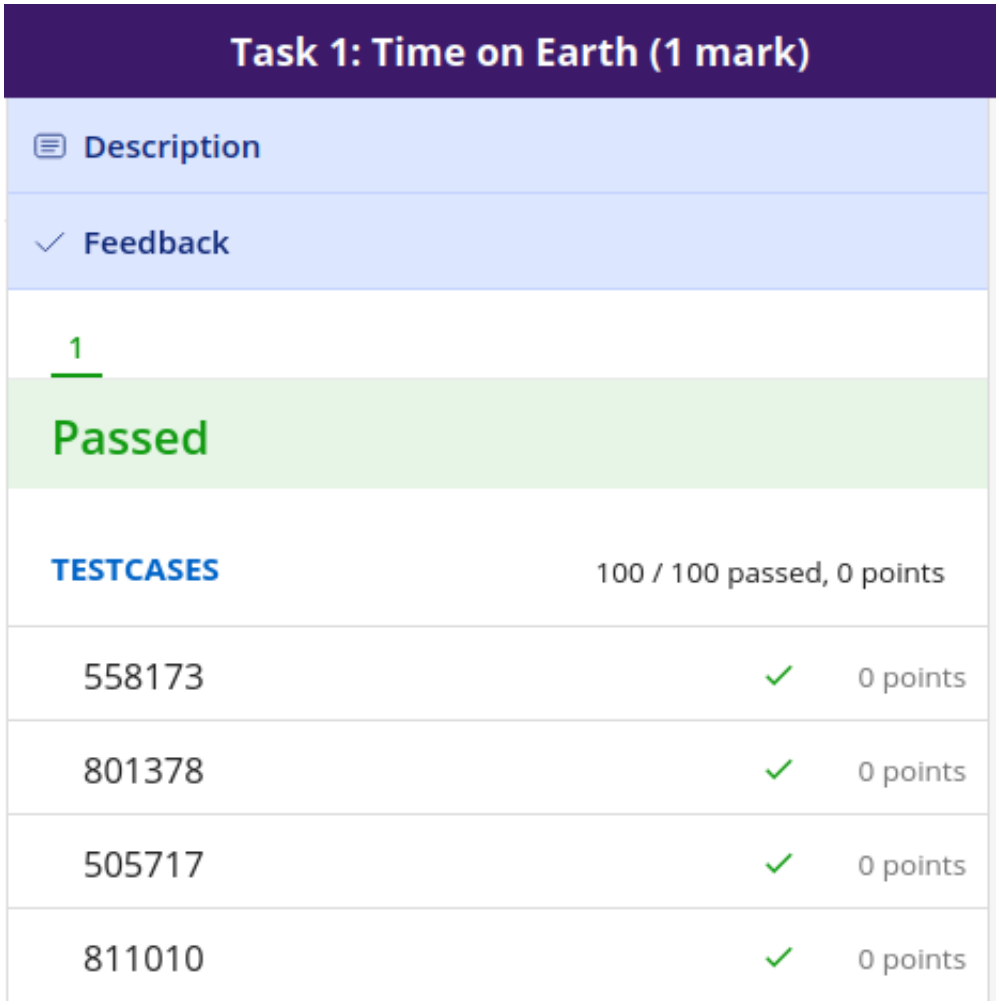
- `.pty` files show the given input and expected output together. These are the most human-readable files.
- `.in` files show the input that is fed to your program.
- `.out` files show the output expected. Beware that some newline characters do not appear because they are in the `.in` file.

Using these files, you can:
- Run the command `python3 name_of_your_program.py < public/558173.in > my_output.out` in the terminal and press Enter to write the output of your program to the file `my_output.out`.
- Run the command `diff my_output.out public/558173.out` in the terminal and press Enter to compare the output of your program to the correct output.

You do not have to do this to complete the assignment, but it may help you on your debugging journey.

Understanding passing test cases

Below is an example of code that passes all 100 **public** tests:



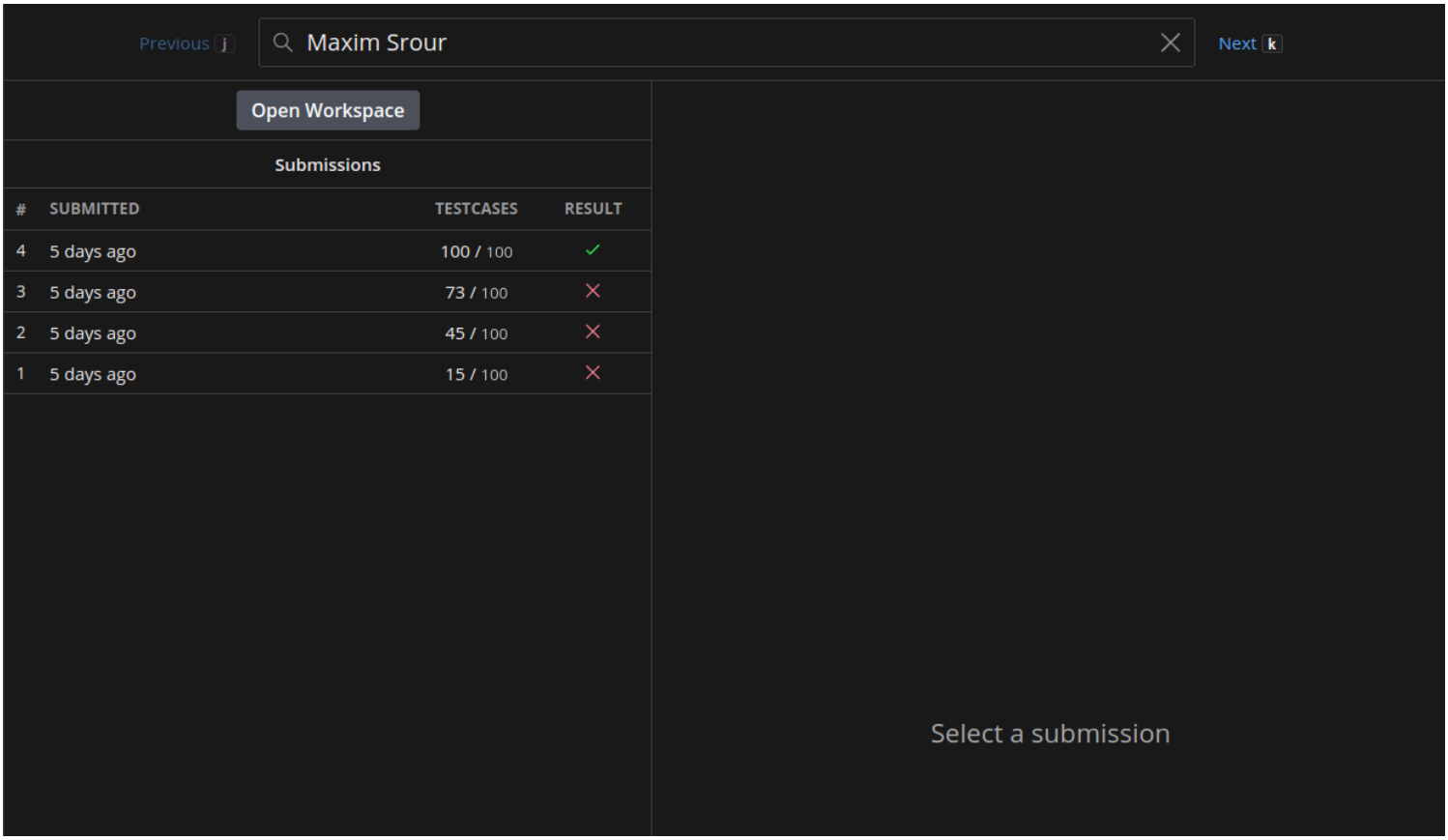
There are two key details to be mindful of. First, is that `100/100 passed` means that this code is passing all 100 of the public tests. If it showed `57/100 passed`, then it would mean only 57 public tests have been passed. Once you pass all the public tests, you will receive a 'green tick' for the whole slide, but this does not necessarily mean that you will pass all private tests and get full marks.

A green tick for the task also has no bearing on the mark you will receive for the quality of the code, as this is assessed manually by the TAs. Make sure that each task conforms to the requirements independently of the number of passing tests.

You will also see the text `0 points` next to every public test and at the top in the summary. This is because public tests are not assessed, hence why they have zero points allocated to them.

Viewing previous Ed submissions

Every time you run the test cases on Ed, it will generate a new 'submission' which we consider for the checkpoint grades. You can see exactly how many test cases you had passed at a particular point in time, as well as review your code for that submission by clicking on it.



If you have made a change to your code and are now failing more test cases, you can use this to review what you had before or to revert back to it entirely.

How will my work be assessed?

Your assignment will be marked based on 3 criteria:

1. **Program quality**, in terms of simplicity and readability, assessed by a TA who will review your code,
2. **Program correctness**, measured by the output of your programs,
3. **Weekly checkpoints**, to reward timely work.

Automated tests

Both criteria 2 and 3 rely on input/output tests of your programs to measure their correctness. This means we test each of your programs with a series of inputs and measure whether they produce the expected output.

For each task, we put 100 public tests at your disposal to help you create correct programs. They are available both in the scaffold and via the "mark" button at the bottom right of each slide:

However, we will use different, private tests, to evaluate your programs. So, passing a given number of public tests is no guarantee you will achieve a specific mark. Therefore, you can use the public tests to guide you, but, first and foremost, make sure your program follows the assignment brief.

1. Program quality

After you submit on Moodle, a TA will review your code and mark its simplicity and readability. This includes:

1. Clear and meaningful variable names,
2. In-line comments (where required),
3. Documentation (at the top of the file),
4. Code clarity.

The review will occur outside of class, after the Moodle submission deadline.

1.1 Variable names

Variable names should be clear. Following the Python variable naming convention will help you achieve clarity. However, this will not be directly assessed.

Variable names should also be meaningful. This means that the name of the variable should tell a human reader what the role of the variable is in the program, including their type.

```
# Bad variable name
x = int(input("Enter your age: "))

# Good variable name
user_age = int(input("Enter your age: "))
```

1.2 In-line comments

You are expected to use in-line comments throughout your program to make sure it is clear to the reader. The amount of comments necessary depends on how clear the code is by itself. Therefore, a good habit is to write clear code, so that the amount of in-line comments required is reduced. This will also reduce the number of bugs created.

Comment your code as you write it rather than after reaching a final solution, as it's a lot easier to remember why certain decisions were made and the thought process behind them when the code is fresh in your mind.

Ensure your variable names are meaningful and concise so that your code is understandable at first read. When a reader is going through your program, it should read like a story that is self-explanatory, using in-line comments to explain the logic behind blocks of code and any additional information that is relevant to the functionality of the code.

Avoid repeating yourself by commenting on lines of code that are self-explanatory, and instead reserve your comments to explain higher level logic, such as explaining the overall purpose behind a small block of code.

```
"""
This is an example of over-commented code.
Each line is already quite self-explanatory.
"""

# This is the first mark
first_mark = float(input("Enter the first mark: "))
# This is the second mark
second_mark = float(input("Enter the second mark: "))
# Computes the average of the two marks
average_mark = (first_mark + second_mark) / 2
# Computes the correctly rounded grade
rounded_grade = int(average_mark + 0.5)
```

1.3 Documentation

You are expected to include documentation at the start of each file you submit. For example:

```
"""
This program reads a name from the user and then greets them.
"""

name = input("What's your name?\n")
print("Greetings,", name)
```

You are also expected to write documentation for every function you define.

```
def name_input() -> str:
    """
    Prompts the user to input their name, and returns it.
    """

    name = input("What's your name?\n")
    return name
```

Documentation should clarify the purpose of a function/file and should provide a high-level overview. Your documentation should not be a rewrite of your comments.

1.4 Code clarity

Optimise your code for readability. In this unit, we do not care about efficiency. So, if the code can be written in multiple ways, opt for the simplest one. A human reader should be able to understand your code quickly and easily.

2. Program correctness

After you submit on Moodle, we will run automated tests on your Moodle submission.

3. Weekly checkpoints

At the end of each week, we expect you to have completed the tasks of the given week.

To measure this, we will look at the percentage of private automated tests passed (i.e. not failed) by your best ed submission made before the weekly checkpoint deadline.

It is therefore necessary for you to run the automated tests on ed if you want to get these marks. You can do so by clicking the "mark" button of each slide.

You are free to edit your code outside ed if you prefer (e.g., a locally installed PyCharm, VSCode, etc), as long as you copy your code on ed to run the tests.

Mark breakdown

The assignment has 25 marks, so 1 mark in the assignment equals one mark in the unit.

- Week 2 (Tasks 1-3):
  - Program quality: 3 marks
  - Task 1 correctness: 1 mark
  - Task 2 correctness: 1 mark
  - Task 3 correctness: 2 marks
  - Weekly checkpoint: 1 mark
  - Subtotal: 8 marks
- Week 3 (Tasks 4-6):
  - Program quality: 3 marks
  - Task 4 correctness: 1 mark
  - Task 5 correctness: 1.5 mark
  - Task 6 correctness: 1.5 marks
  - Weekly checkpoint: 1 mark
  - Subtotal: 8 marks
- Week 4 (Tasks 7-9):
  - Program quality: 3 marks
  - Task 7 correctness: 2 mark
  - Task 8 correctness: 2 mark
  - Task 9 correctness: 1 mark
  - Subtotal: 8 marks
- Correctly formatted Moodle submission: 1 mark
- Total: 25 marks

Program quality

Maximum achievable mark

A maximum of 3 marks can be achieved for program quality for each set of 3 tasks.

This is further capped depending on how much of these 3 tasks you have completed.

This is measured by the fraction of the private tests that your programs pass for these 3 tasks.

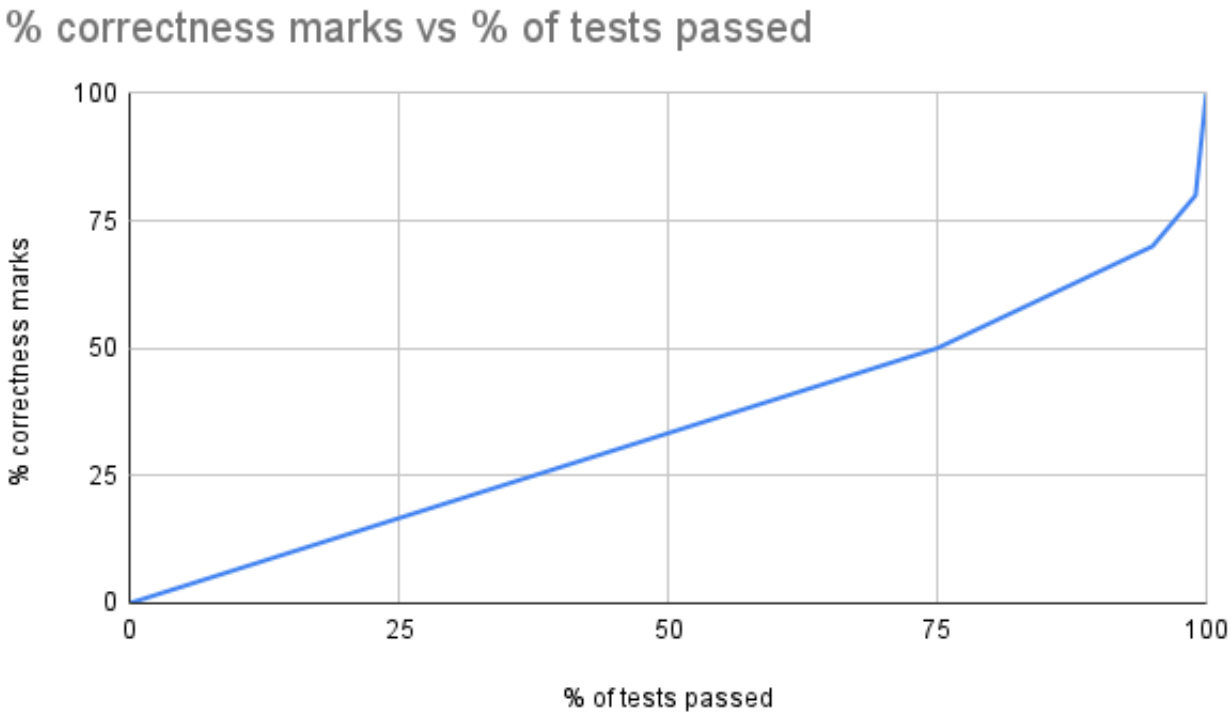
For instance, if your programs pass 100% of the private tests on task 1, 80% of the tests on task 2 and 30% of the tests on task 3, the quality mark is capped at (100 + 80 + 30)/300 * 3 = 2.1 marks.

Program correctness

We will evaluate each program with private tests. If your program passes

- 100% of these tests, it receives 100% of the correctness marks,
- 95% to 99% of these tests, it receives 70 to 79% of the correctness marks,
- 75% to 95% of these tests, it receives 50 to 70% of the correctness marks,
- 0% to 75% of these tests, it receives 0 to 50% of the correctness marks.

See chart below.



The rationale is that getting the first 50 tests right is quite easier than getting the last 50 tests right. Furthermore, a program that passes 100% of the tests is more than 1% better than a program that passes 99% of the tests.

Weekly checkpoint

Weekly checkpoint marks are subject to the same curve as program correctness marks.

Penalties

Late penalties

- Up to 7 days: 5% lost per calendar day (or part thereof).
- More than 7 days: Zero (0) marks with no feedback given.

Late penalties are determined using the submission date on Moodle.

Use of external modules

You are allowed to import modules if they are part of the Python Standard Library (e.g. math), but not other modules (e.g. numpy). However, note that the solutions do not require the use of modules.

If your solution to a task uses external modules, it will void (i.e. set to 0) any mark it may have received for this task from program correctness, program quality and weekly checkpoint.

Marking rubric

This page is intentionally left blank.

Task 1-3 quality (3 marks)

Write quality code from the get-go.

See how in the slide "How will my work be assessed?".

Task 1: Time on Earth (1 mark)

Task Description

Write a program that converts a number of seconds into seconds, minutes, and hours.

> ⚠ For Tasks 1-3,
> - Assume all inputs are positive integers, and
> - We write all units in plural, regardless of the quantity (e.g. `0 hours`, `1 seconds`).

Input and Output Examples

User inputs are in **bold font** below.

Example 1

```
TIME ON EARTH
Input a time in seconds:
121

The time on Earth is 0 hours 2 minutes and 1 seconds.
```

Example 2

```
TIME ON EARTH
Input a time in seconds:
3600

The time on Earth is 1 hours 0 minutes and 0 seconds.
```

Example 3

```
TIME ON EARTH
Input a time in seconds:
123456

The time on Earth is 34 hours 17 minutes and 36 seconds.
```

Task 2: Time on Trisolaris (1 mark)

This task is similar to Task 1, but the rate of conversion between units is not fixed.

Task Description

Extend your previous program so that it now also converts a time given in seconds into seconds, minutes, and hours, where the conversion from one unit to the other is defined during the program, rather than being fixed.

Input and Output Examples

User inputs are in **bold font** below.

Example 1

```
TIME ON EARTH
Input a time in seconds:
121

The time on Earth is 0 hours 2 minutes and 1 seconds.

TIME ON TRISOLARIS
Input the number of seconds in a minute on Trisolaris:
10
Input the number of minutes in an hour on Trisolaris:
10

The time on Trisolaris is 1 hours 2 minutes and 1 seconds.
```

Example 2

```
TIME ON EARTH
Input a time in seconds:
3600

The time on Earth is 1 hours 0 minutes and 0 seconds.

TIME ON TRISOLARIS
Input the number of seconds in a minute on Trisolaris:
60
Input the number of minutes in an hour on Trisolaris:
60

The time on Trisolaris is 1 hours 0 minutes and 0 seconds.
```

Example 3

```
TIME ON EARTH
Input a time in seconds:
123456

The time on Earth is 34 hours 17 minutes and 36 seconds.

TIME ON TRISOLARIS
Input the number of seconds in a minute on Trisolaris:
123
Input the number of minutes in an hour on Trisolaris:
456

The time on Trisolaris is 2 hours 91 minutes and 87 seconds.
```

Task 3: Time addition on Trisolaris (2 marks)

This task uses the same unit conversion as Task 2, but this time we're adding times!

Task Description

Extend the previous program to also read a duration in seconds, then add the two times and display the resulting time.

Input and Output Examples

User inputs are in **bold font** below.

Example 1

```
TIME ON EARTH
Input a time in seconds:
121

The time on Earth is 0 hours 2 minutes and 1 seconds.

TIME ON TRISOLARIS
Input the number of seconds in a minute on Trisolaris:
10
Input the number of minutes in an hour on Trisolaris:
10

The time on Trisolaris is 1 hours 2 minutes and 1 seconds.

TIME AFTER WAITING ON TRISOLARIS
Input a duration in seconds:
61

The time on Trisolaris after waiting is 1 hours 8 minutes and 2 seconds.
```

Example 2

```
TIME ON EARTH
Input a time in seconds:
3600

The time on Earth is 1 hours 0 minutes and 0 seconds.

TIME ON TRISOLARIS
Input the number of seconds in a minute on Trisolaris:
60
Input the number of minutes in an hour on Trisolaris:
60

The time on Trisolaris is 1 hours 0 minutes and 0 seconds.

TIME AFTER WAITING ON TRISOLARIS
Input a duration in seconds:
60

The time on Trisolaris after waiting is 1 hours 1 minutes and 0 seconds.
```

Example 3

```
TIME ON EARTH
Input a time in seconds:
123456

The time on Earth is 34 hours 17 minutes and 36 seconds.

TIME ON TRISOLARIS
Input the number of seconds in a minute on Trisolaris:
123
Input the number of minutes in an hour on Trisolaris:
456

The time on Trisolaris is 2 hours 91 minutes and 87 seconds.

TIME AFTER WAITING ON TRISOLARIS
Input a duration in seconds:
9001

The time on Trisolaris after waiting is 2 hours 164 minutes and 109 seconds.
```

Week 2 checkpoint (1 mark)

The tasks of week 2 are Tasks 1, 2 and 3.

The checkpoint is Friday, August 2, 2024, 23:55 (Melbourne time), 21:55 (Malaysia time).

This page is intentionally left blank.

Task 4-6 quality (3 marks)

Write quality code from the get-go.

See how in the slide "How will my work be assessed?".

Task 4: Username login (1 mark)

Task Description

Write a program that asks the user to enter a username. A successful login occurs when the user enters one of the valid usernames. The program keeps prompting the user until a valid username is provided.

The usernames are given below. They are case-sensitive.

```
Usernames
+----+----+----+----+----+----+----+----+----+----+
| Ava| Leo| Raj| Zoe| Max| Sam| Eli| Mia| Ian| Kim|
+----+----+----+----+----+----+----+----+----+----+
```

Input and Output Examples

User inputs are in **bold font** below.

Example 1

```
Enter username: Raj
Login successful. Welcome Raj !
```

Example 2

```
Enter username: Rio
Login incorrect.
Enter username: Ivy
Login incorrect.
Enter username: Sam
Login successful. Welcome Sam !
```

Task 5: Passwords & tries (1.5 marks)

This task is similar to Task 4, but:

- *A password must also be entered,*
- *After 3 unsuccessful tries, the program terminates.*

Task Description

Write a program that asks the user to enter a valid username and a valid password. A successful login occurs when the user enters one of the correct usernames and one of the correct passwords. If, after 3 tries, the user did not provide a correct login, the program terminates.

> Given the description above, the program can either terminate with a successful login or after 3 unsuccessful attempts.

The usernames and passwords are given below. They are case-sensitive.

```
Usernames
+----+----+----+----+----+----+----+----+----+----+
| Ava| Leo| Raj| Zoe| Max| Sam| Eli| Mia| Ian| Kim|
+----+----+----+----+----+----+----+----+----+----+
```

```
Passwords
+--------+--------+--------+--------+--------+--------+--------+--------+--------+----
| 12345  | abcde  | pass1  | qwert  | aaaaa  | zzzzz  | 11111  | apple  | hello  | adm
+--------+--------+--------+--------+--------+--------+--------+--------+--------+----
```

Input and Output Examples

User inputs are in **bold font** below.

Example 1

```
Enter username: Raj
Enter password: pass1
Login successful. Welcome Raj !
```

Example 2

```
Enter username: Ian
Enter password: 1234
Login incorrect. Tries left: 2
Enter username: Ava
Enter password: pass1
Login successful. Welcome Ava !
```

Example 3

```
Enter username: Rio
Enter password: opqr
Login incorrect. Tries left: 2
Enter username: Ivy
Enter password: pass1
Login incorrect. Tries left: 1
Enter username: Ona
Enter password: aaaaa
Login incorrect. Tries left: 0
```

Task 6: Matching login & robot check (1.5 marks)

This task is similar to Task 5, but:

- *The username and password must match, and*
- *After 3 unsuccessful tries, the user is asked to confirm that they are not a robot so they can try to log in again.*

Task Description

Write a program that asks the user to enter a valid login. In this task, a valid login is a pair of username/password. Each username has a single password associated to it. If, after 3 tries, the user did not provide a correct login, the program asks whether they are a robot. If they are not a robot, the user is allowed 3 more tries. If they are a robot, the program terminates.

When asked if they are a robot, the prompt reads `"Are you a robot (Y/n)?"`. If the user inputs `"Y"` or nothing (i.e. `""`), then the program terminates. If the user enters `"n"`, they are given 3 more tries to log in. If the user enters anything else, the prompt `"Are you a robot (Y/n)?"` is asked again and the user must answer again.

In this task, you may need to:
- use a `while` loop inside another `while` loop,
- use lists of lists, and
- use `==` to compare two lists.

Given the description above, the program can only terminate upon a succesful login or a failed robot check.

The pairs of usernames and passwords are given below. They are case-sensitive.

```
|----------|----------|
| Username | Password |
|----------|----------|
| Ava      | 12345    |
| Leo      | abcde    |
| Raj      | pass1    |
| Zoe      | qwert    |
| Max      | aaaaa    |
| Sam      | zzzzz    |
| Eli      | 11111    |
| Mia      | apple    |
| Ian      | hello    |
| Kim      | admin    |
|----------|----------|
```

Because we are asking for a matching pair, inputting username "Ava" with password "abcde", for example, results in a failed login attempt, even though both strings are in this table. Only password "12345" leads to a succesful login for username "Ava".

Input and Output Examples

User inputs are in **bold font** below.

Example 1

```
Enter username: Lux
Enter password: 12345
Login incorrect. Tries left: 2
Enter username: Kim
Enter password: pass
Login incorrect. Tries left: 1
Enter username: Ravi
Enter password: pass
Login incorrect. Tries left: 0
Are you a robot (Y/n)? lol
Are you a robot (Y/n)? Y
```

Example 2

```
Enter username: Ava
Enter password: zzzzz
Login incorrect. Tries left: 2
Enter username: Leo
Enter password: aaaa
Login incorrect. Tries left: 1
Enter username: Ian
Enter password: opqr
Login incorrect. Tries left: 0
Are you a robot (Y/n)? gg ez
Are you a robot (Y/n)? n
Enter username: Zoe
Enter password: zzzzz
Login incorrect. Tries left: 2
Enter username: Zoe
Enter password: qwert
Login successful. Welcome Zoe !
```

Example 3

```
Enter username: Leo
Enter password: qwer
Login incorrect. Tries left: 2
Enter username: Eli
Enter password: abcd
Login incorrect. Tries left: 1
Enter username: Lux
Enter password: aaaaa
Login incorrect. Tries left: 0
Are you a robot (Y/n)? gg ez
Are you a robot (Y/n)? quit
Are you a robot (Y/n)? n
Enter username: Ona
Enter password: wxyz
Login incorrect. Tries left: 2
Enter username: Ava
Enter password: opqr
Login incorrect. Tries left: 1
Enter username: Raj
Enter password: 4321
Login incorrect. Tries left: 0
Are you a robot (Y/n)? lol
Are you a robot (Y/n)? n
Enter username: Mia
Enter password: wxyz
Login incorrect. Tries left: 2
Enter username: Ian
Enter password: zxcv
Login incorrect. Tries left: 1
Enter username: Ravi
Enter password: pass1
Login incorrect. Tries left: 0
Are you a robot (Y/n)? lol
Are you a robot (Y/n)?
```

At the last line of the example above, the user pressed enter without entering anything.

Week 3 checkpoint (1 mark)

The tasks of week 3 are Tasks 4, 5 and 6.

The checkpoint is Friday, August 9, 2024, 23:55 (Melbourne time), 21:55 (Malaysia time).

This page is intentionally left blank.

Task 7-9 quality (3 marks)

Write quality code from the get-go.

See how in the slide "How will my work be assessed?".

Hints for Task 7

If you do not know how to get started with Task 7, we recommend you decompose the problem into the following steps:

1. Finding the coordinates of a letter in a keyboard

Complete the code below (with meaningful variable names you will choose).

```
your_var_name = \
["abcdefghijklm",
 "nopqrstuvwxyz"]

your_var_name_2 = input("Enter a character to find the position of: ")

# write your code here (8-10 lines should suffice)

print(your_var_name_2 + " is on row " + str(your_var_name_3) + " and column " + str(yo
```

Verify with a few examples that the program prints the right values. For example,

- for input `'a'`, the output should be `"a is on row 0 and column 0"`,
- for input `'n'`, the output should be `"n is on row 1 and column 0"`,
- for input `'r'`, the output should be `"r is on row 1 and column 4"`.

2. Finding the moves between two locations

Complete the code below (with meaningful variable names you will choose).

```
x1 = 0
x2 = 0

# write your code here (5-8 lines should suffice)

print("horizonal moves: " + your_var_name_5)
```

Verify with a few examples that the program prints the right values. For example,

- for values `x1=0` and `x2=0`, the output should be `"horizonal moves: "`,
- for values `x1=0` and `x2=1`, the output should be `"horizonal moves: r"`,
- for values `x1=4` and `x2=1`, the output should be `"horizonal moves: lll"`,

Once this works, asks yourself how this logic would translate to computing vertical moves.

3. Putting the two programs together

Combine the two programs above to print the moves required to go from one position in the keyboard to a letter (from which you deduce a position).

4. Looping through the characters of the input

Now that you have a program that works for two letters, you need to repeat the procedure for successive letters in the input word. And make sure you keep track of Robbie's current position!

Task 7: One config (2 marks)

POV: you are Robbie the robot using a keyboard. What signal do you send your bionic fingers to type the words?

Task Description

Write a program that asks the user to input a string, and then plans the actions of Robbie the robot so that it can type this string on a keyboard.

The keyboard has the layout below.

```
abcdefghijklm
nopqrstuvwxyz
```

Robbie starts at the top left position (a) and can take the following actions:

```
u - go up
d - go down
l - go left
r - go right
p - press the key
```

After pressing a key, Robbie stays on that key. This means:

- It can immediately press the same key again.
- To go to another key, Robbie will start from the last key it moved to.

Furthermore, Robbie will always move horizontally (left/right) before moving vertically (up/down) when moving to a key it needs to press.

Robbie's actions are restricted by the limits of the board:

- It cannot take an action that would make it leave the board.
- It cannot wrap around the board.

Any key not on the keyboard cannot be typed.

We recommend you store the key configuration of the keyboard as a list of strings. See example below.

```
your_variable_name = \
["abcdefghijklm",
 "nopqrstuvwxyz"]
print(your_variable_name[0])
print(your_variable_name[1][2])
```

Input and Output Examples

User inputs are in **bold font** below.

Example 1

```
Enter a string to type: k
The robot must perform the following operations:
rrrrrrrrrp
```

Example 2

```
Enter a string to type: .716
The string cannot be typed out.
```

Example 3

```
Enter a string to type: helpiamsentient
The robot must perform the following operations:
rrrrrrrplllprrrrrrrplllllllldprrrrrruplllllllprrrrrrrrrrrplllllldpluplllldprrrrrrp
```

Task 8: Many configs (2 marks)

This task uses the same movement rules as Task 7, but this time Robbie has multiple keyboard configurations to choose from, and will choose the one that requires the fewest moves.

Task Description

Write a program that asks the user to input a string, then choose a keyboard, then plans the actions of Robbie the robot so that it can type this string on a keyboard.

For a given input string,

- If there is a single keyboard on which in can be typed, then Robbie picks that one.
- If it can be typed on multiple keyboards, Robbie picks a single keyboard as follows:
  - The one that requires the fewest moves, or, if there is a tie,
  - The first best keyboard configuration (in the order we give them).
- Finally, there may not be a keyboard that can type that string.

The keyboards have the configurations below.

Configuration 0

```
abcdefghijklm
nopqrstuvwxyz
```

Configuration 1

```
789
456
123
0.-
```

Configuration 2

```
chunk
vibex
gymps
fjord
waltz
```

Configuration 3

```
bemix
vozhd
grypt
clunk
waqfs
```

Robbie starts at the top left position of the chosen keyboard.

Input and Output Examples

User inputs are in **bold font** below.

Example 1

```
Enter a string to type: k
Configuration used:
---------
| chunk |
| vibex |
| gymps |
| fjord |
| waltz |
---------
The robot must perform the following operations:
rrrrp
```

Example 2

```
Enter a string to type: .716
Configuration used:
-------
| 789 |
| 456 |
| 123 |
| 0.- |
-------
The robot must perform the following operations:
rdddpluuupddprrup
```

Example 3

```
Enter a string to type: y
Configuration used:
---------
| chunk |
| vibex |
| gymps |
| fjord |
| waltz |
---------
The robot must perform the following operations:
rddp
```

Example 4

```
Enter a string to type: (ﾉ°□°)ﾉ⌒ ┻┻
The string cannot be typed out.
```

Task 9: Many configs & wrap (1 mark)

This task is similar to Task 8, but this time Robbie can "wrap" around the keyboard, which means cross the border to the other side of the keyboard.

Task Description

Write a program that asks the user to input a string, then choose a keyboard, then plans the actions of Robbie the robot so that it can type this string on a keyboard.

This time, though, Robbie is able to wrap around the keyboard. For example:

- In the row "abcde", Robbie can move from 'a' to 'e' in 1 move to the left. This move is encoded as "lw", where "w" marks a wrap (horizontal or vertical).
- In the row "abcde", Robbie can move from 'e' to 'a' in 1 move to the right. This move is encoded as "rw".
- The same applies for columns. The character 'w' is also used to mark vertical wrapping.
- Proud of this new technique, and for style points, Robbie uses wrapping whenever it does not increase the number of moves required. This means that, for example, in the row "ab", to go from 'a' to 'b', Robbie will always prefer doing "lw" than "r" ('w' does not count as a move).

The other rules and keyboard configurations are the same as in Task 8.

For a given input string, the best keyboard may not be the same as for Task 8, now that wrapping is possible.

Input and Output Examples

User inputs are in **bold font** below.

Example 1

```
Enter a string to type: k
Configuration used:
---------
| chunk |
| vibex |
| gymps |
| fjord |
| waltz |
---------
The robot must perform the following operations:
lwp
```

Example 2

```
Enter a string to type: .716
Configuration used:
-------
| 789 |
| 456 |
| 123 |
| 0.- |
-------
The robot must perform the following operations:
ruwpldwpuwuplwup
```

Example 3

```
Enter a string to type: y
Configuration used:
-----------------
| abcdefghijklm |
| nopqrstuvwxyz |
-----------------
The robot must perform the following operations:
lwluwp
```

Example 4

```
Enter a string to type: (ノ°□°)ノ︵ ┻━┻
The string cannot be typed out.
```

Final submission on Moodle (1 mark)

Your final submission must be one zip file with the necessary and sufficient files, as shown below.

```
A1_<student_id>.zip
└─> login_match_robot.py
└─> login_password_tries.py
└─> login_username.py
└─> time_addition.py
└─> time_on_earth.py
└─> time_on_trisolaris.py
└─> typing_robot_many_configs.py
└─> typing_robot_many_configs_with_wrap.py
└─> typing_robot_one_config.py
```

Alternatively, you may also submit in the following format.

```
A1_<student_id>.zip
└─> A1_<student_id>
    └─> login_match_robot.py
    └─> login_password_tries.py
    └─> login_username.py
    └─> time_addition.py
    └─> time_on_earth.py
    └─> time_on_trisolaris.py
    └─> typing_robot_many_configs.py
    └─> typing_robot_many_configs_with_wrap.py
    └─> typing_robot_one_config.py
```

If your submission follows the structures above, you will receive one mark. Make sure that you replace `<student_id>` with your actual student ID, and ensure that you use the correct names for each of the files, including the file extension.

For more information please check the Ed post

⚠ Some operating systems hide the file extension, so be sure to check if that's the case before "adding it back in".

This page is intentionally left blank.

Changelog (9 changes)

Change 9 - Thursday 29 August

Updated the fraction of marks available in the marking rubric for program quality. Instead of Excellent being worth 100%, it is now correctly listed as "up to 100%". This is due to the fact that the program quality grade is capped by the number of passing tests for the tasks (see the slide "Mark breakdown" for more information about this process).

Change 8 - Friday 16 August

Added the link on 'Final submission on Moodle' slide for submission instruction

> For more information please check the Ed post

Change 7 - Wednesday 14 August

Added the slide Hints for Task 7.

Change 6 - Monday 12 August

Added a new slide titled, "Marking rubric" which contains a full table of all possible marks available.

Change 5 - Friday 2 August

In the slide "Tips on working with automated tests", added a section "Viewing previous Ed submissions" with information on how to find your prior submissions.

Change 4 - Tuesday 30 July

In Task 3, the three examples given on the slides were missing the string "and " in the last line of the output. They have now been added:

```
The time on Trisolaris after waiting is 1 hours 8 minutes and 2 seconds.
```

```
The time on Trisolaris after waiting is 1 hours 1 minutes and 0 seconds.
```

```
The time on Trisolaris after waiting is 2 hours 164 minutes and 109 seconds.
```

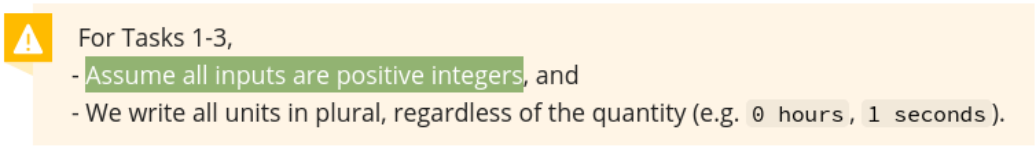
Thank you Jiun for reporting this issue. See this post.

Change 3 - Monday 29 July

In the slide "Tips on working with automated tests", added a heading "Understanding passing test cases" which clarifies what points are and what it means to pass public tests.

Change 2 - Monday 29 July

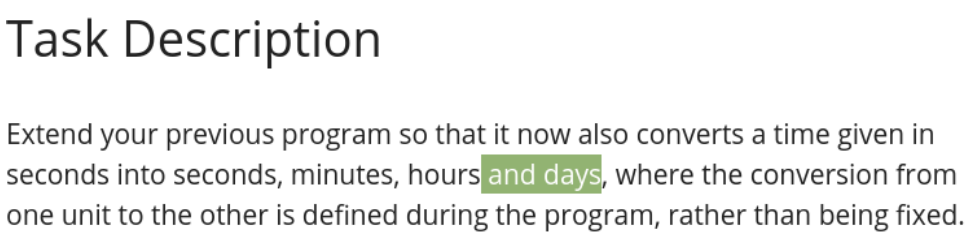
In Task 1, we made explicit the assumption that all inputs are positive integers for Tasks 1-3.



Cheers to Prabhisher and Ibrahim for raising the question. See this post.

Change 1 - Monday 29 July

In the description of Task 2, removed "and days".



Cheers to Muhamad for pointing this out. See this post.